`A13` Componentes

📕 TextView

Este componente exibe uma mensagem de texto para o usuário [1].

```
<TextView
    android:id="@+id/text_view_id"
    android:layout_height="wrap_content"
    android:layout_width="wrap_content"
    android:text="@string/hello" />
```

📙 TextInput

O `TextInputLayout` esconde o `hint` do `TextInputEditText` interno quando o usuário digita um texto. Esse layout suporta várias ações que tornam a caixa de entrada de texto mais amigável [2].

```
<com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/form_username">

    <com.google.android.material.textfield.TextInputEditText
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>

</com.google.android.material.textfield.TextInputLayout>
```

Algumas funcionalidade suportadas [2]:

- `setErrorEnabled(boolean)`, `setError(CharSequence)`, `setErrorIconDrawable(Drawable)`: exibe um ícone e uma mensagem de erro. É usado para va-

lidação do campo.

- `setHelperTextEnabled(boolean)` e `setHelperText(CharSequence)`: exibe uma mensagem de ajuda para o capo.
- `setCounterEnabled(boolean)` e `setCounterMaxLength(int)`: exibe um contador de caracteres.
- `setEndIconMode(int)`: exibe um botão no final do campo que pode ser usado para exibir o texto se for um campo de password.

📗 Checkbox

Este componente permite que o usuário selecione mais de uma opção de um conjunto. O evento `setOnClickListener()` captura quando o componente foi clicado. O método `isChecked` retorna `true` se o componente está selecionado, e `false` caso contrário. Para alterar o valor do componente via código é possível usar o método `setChecked(boolean)` ou `toogle()` [3].

```
<CheckBox android:id="@+id/checkbox"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/check1"/>
```

📘 RadioButton

Este componente é semelhante ao `Checkbox`, no entanto, não pode ser desselecionado após o usuário selecioná-lo. Os botões que estiverem no mesmo `RadioGroup` são mutualmente exclusivos [4].

```
<RadioGroup
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">
    <RadioButton android:id="@+id/radio_pirates"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/pirates"/>
    <RadioButton android:id="@+id/radio_ninjas"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/ninjas"/>
</RadioGroup>
```

📕 Switch

Este componente é semelhante ao `Checkbox`, no entanto ele é exibido como uma alavanca [5].

```
<Switch android:id="@+id/switch"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/switch1"/>
```

📙 Button

O botão é um componente que o usuário pressiona para realizar uma ação. O evento `setOnClickListener()` captura o evento de clique [6]

```
 <Button
     android:id="@+id/button_id"
     android:layout_height="wrap_content"
     android:layout_width="wrap_content"
     android:text="@string/self_destruct" />
```

```
button.setOnClickListener { v ->
    // código do evento vem aqui
}
```

📗 Acessando componentes via código

A referência aos componentes de layout via código pode ser feita de duas maneiras: (1) `findViewById()` ou (2) `DataBinding`. Em ambos os modos, a referência só é possível se o atributo `android:id` for informado no arquivo XML. A maneira mais tradicional de recuperar um componente é usando o método `findViewById()`.

```
var texto = findViewById(R.id.txt_texto)
texto.setText("Texto alterado")
```

O problema dessa abordagem é que se o `id` de um componente for alterado, ou atribuído a outro componente, o problema só será detectado em tempo de execução. Isso pode tornar a identificação de bugs mais trabalhosa.

A segunda maneira é utilizando o `DataBinding` que gera uma classe que representa o arquivo de layout e já contém as referências a todos os componentes. Caso algum `id` seja alterado, a classe é gerada novamente e erros são capturados em tempo de compilação. Para utilizar o `DataBinding` é necessário realizar os seguintes passos:

- Habilitar o `DataBinding` no arquivo `build.gradle`.

```
plugins {
        ...
    id 'kotlin-android-extensions' // Incluir o plugin
}

android {
    ...
    buildFeatures {
    viewBinding true  // Habilitar o data binding
    }
}
```

- Feito isso, para cada arquivo de layout será criada uma classe de `binding` (amarração). Por exemplo, para o arquivo `activity_aluno.xml` será criada a classe `ActivityAlunoBinding`.

Com a utilização do `DataBinding` será necessário mudar o processo de definição dos layouts das atividades da seguinte maneira:

```
override fun onCreate(savedInstanceState: Bundle?) {
   super.onCreate(savedInstanceState)

   binding = ActivityAlunoBinding.inflate(layoutInflater)
   setContentView(binding.root)
}
```

Os componentes podem ser acessados diretamente a partir do objeto de `binding` diretamente:

```
    binding.btnConfirmar.text = "OK"
    binding.checkbox.toogle()
```

📚 Referência bibliográficas

[1] Textview. **Android Developers**. Acessado em 14/01/2021.

[2] TextInputLayout. **Android Developers**. Acessado em 14/01/2021.

[3] Checkbox. **Android Developers**. Acessado em 14/01/2021.

[4] RadioButton. **Android Developers**. Acessado em 14/01/2021.

[5] Switch. **Android Developers**. Acessado em 14/01/2021.

[6] Button. **Android Developers**. Acessado em 14/01/2021.